

Processo de Desenvolvimento com IA

PROCESSO DE DESENVOLVIMENTO COM IA — RADAR / PROJETOS QUANTITATIVOS

Versão 1.0 — 08/09/2026 **Autor:** Auditoria de Sistemas Sênior **Aplicação:** correção do RADAR-TASYTRADE (Parte B) e todo desenvolvimento novo (Parte A) **Documento de referência:** AUDITORIA_RADAR_TASTYTRADE.pdf

SUMÁRIO EXECUTIVO — LEIA ISTO SE LER SÓ UMA PÁGINA

O sistema auditado não falhou por falta de regra. **As regras existiam nas suas cinco skills e foram violadas mesmo assim.** Portanto o processo abaixo é construído sobre um princípio diferente:

Regra escrita é conselho. Barreira automática é regra. Todo controle deste processo tem uma verificação mecânica associada — ou não é um controle, é uma intenção.

O processo tem quatro alavancas, em ordem decrescente de eficácia:

#	Alavanca	Por que funciona	Custo
1	CLAUDE.md no projeto	Carrega em toda sessão, sem gatilho. Skill só dispara se a descrição casar com o pedido — invariante não pode depender disso.	1 h, uma vez
2	Tipo + lint + CI	A IA não consegue escrever o defeito porque o compilador rejeita. Não depende de ela lembrar.	meio dia, uma vez
3	Portões com artefato de saída	Cada etapa produz um documento verificável em minutos. Sem o artefato, a etapa não terminou.	por tarefa
4	Skills	Dão vocabulário, checklist e profundidade técnica. Necessárias, mas as mais fracas isoladamente.	já existem

Você tem hoje a alavanca 4 e nada das outras três. É exatamente o resultado observado na auditoria.

PARTE A — O PROCESSO PADRÃO (DESENVOLVIMENTO NOVO)

Visão geral do pipeline

```
G0 FUNDAÇÃO (uma vez por projeto)
  ↳ CLAUDE.md + tipos marcados + lint + CI
  |
G1 REQUISITO —————> skill: analista-funcional-requisitos
  | saída: documento de requisito + critérios de aceite
  ▼
G2 PLANO TÉCNICO —————> skill: papel-engenheiro-senior
  | saída: plano + criticidade + APROVAÇÃO SUA (portão humano)
  ▼
G3 CONTRATO DE DADO ———> skill: integridade-de-dado-exibido
  | saída: Mapa de Proveniência + PROVA DE FONTE ← portão que teria pego tudo
  ▼
G4 CONSTRUÇÃO —————> skill: padroes-engenharia-software
  | saída: código em fatias pequenas, uma por vez
  ▼
G5 VALIDAÇÃO —————> skill: qa-sistemas-proprios + integridade-de-dado-exibido
  | saída: golden tests com valor calculado à mão
  ▼
G6 DEPLOY —————> skill: arquitetura-infraestrutura-deploy
  | saída: pipeline verde + plano de rollback
  ▼
G7 AUDITORIA —————> SESSÃO NOVA, CONTEXTO LIMPO, POSTURA ADVERSARIAL
  saída: laudo independente
```

Regra de ouro do fluxo: nenhum portão avança sem o artefato de saída do anterior. Não é burocracia — é o que torna a revisão possível em minutos em vez de exigir auditoria de 10 mil linhas.

G0 — FUNDAÇÃO (uma vez por projeto, antes da primeira linha)

Este é o portão de maior retorno de todo o processo. Sem ele, os demais degradam sob pressão de prazo.

G0.1 — Criar o `CLAUDE.md` na raiz do projeto

Por que é a alavanca nº 1: skill precisa de gatilho — a descrição tem que casar com o que você pediu. Se você escreve "*ajusta o layout do card de volatilidade*", nenhuma skill de integridade de dado dispara, e a IA fica livre para preencher um campo vazio com preset no meio do ajuste de layout. O `CLAUDE.md` não tem gatilho: **ele entra em toda sessão, em todo pedido, inclusive nos pequenos.** É onde moram os invariantes que não podem depender de sorte.

Modelo pronto em **Anexo I**.

G0.2 — Tipo marcado de proveniência

```
// src/lib/types/provenance.ts
export type Source = 'MEDIDO' | 'DERIVADO' | 'ESTIMADO' | 'SIMULADO';

export interface Tracked<T> {
  readonly value: T;
  readonly source: Source;
  readonly asOf: string; // ISO 8601
  readonly origin: string; // 'tastytrade:/market-metrics', 'brapi:/quote'
}

/** Só o adaptador de borda pode criar isto. Domínio consome, nunca fabrica. */
export function measured<T>(value: T, origin: string): Tracked<T> {
  return { value, source: 'MEDIDO', asOf: new Date().toISOString(), origin };
}
```

O que isso muda na prática: `spot: Tracked<number>` não aceita `142.50`. O literal não compila. A IA não deixa de fabricar porque foi instruída — ela não consegue expressar a fabricação. É a diferença entre pedir e impedir.

G0.3 — Lint que barra o padrão de falha

```
// .eslintrc.json - regras mínimas
{
  "overrides": [{
    "files": ["src/lib/domain/**/*.ts", "src/lib/services/**/*.ts"],
    "rules": {
      "no-restricted-syntax": ["error",
        { "selector": "CallExpression[callee.object.name='Math'][callee.property.name='random']",
          "message": "Math.random() proibido em dominio/servico. Dado sintetico nao entra em motor de calculo." },
        { "selector": "CatchClause[body.body.length=0]",
          "message": "catch vazio proibido. Falha de fonte retorna null + motivo, nunca valor default." }
      ],
      "no-magic-numbers": ["warn", { "ignore": [0, 1, -1, 2, 100], "enforceConst": true }]
    }
  ]
}
```

G0.4 — CI que bloqueia merge

```
# .github/workflows/ci.yml
name: CI
on: [push, pull_request]
jobs:
  gate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20', cache: 'npm' }
```

```
- run: npm ci
- run: npx tsc --noEmit          # tipagem
- run: npm run lint              # padroes proibidos
- run: npm run test -- --coverage # golden tests
- run: npm run build             # build real
```

E **aposente o atualizar_github.bat**. Commit automático com mensagem de timestamp destrói o valor de auditoria do histórico: hoje 3 dos 9 commits do RADAR são "Update: Atualizacao do Radar <data>" e não dizem o que mudou.

Critério de saída do G0

- ☐ CLAUDE.md na raiz, preenchido
- ☐ provenance.ts criado e usado em pelo menos um fluxo
- ☐ npm run lint falha se alguém escrever Math.random() em domain/
- ☐ CI vermelho bloqueia merge (teste isso de propósito uma vez)

G1 — REQUISITO

Skill: analista-funcional-requisitos **Artefato de saída:** documento de requisito com critérios de aceite mensuráveis **Quem aprova:** você

Não pule esta etapa em feature pequena — pule o *formalismo*, não a etapa. Cinco linhas respondendo as cinco perguntas já basta em tarefa simples.

O que precisa estar respondido, por escrito, antes de sair daqui:

1. Que problema real resolve (não a tela pedida — o objetivo)
2. Qual a regra de negócio, em linguagem de negócio
3. Quais as exceções
4. O que acontece quando falha
5. **De onde vem cada dado que a tela vai mostrar** ← acréscimo obrigatório em sistema financeiro

O item 5 não está na skill original e é o que teria evitado 7 dos 7 achados críticos da auditoria.

Prompt: ver Anexo II, Prompt 1.

G2 — PLANO TÉCNICO E CRITICIDADE

Skill: papel-engenheiro-senior **Artefato de saída:** plano numerado, com classificação de criticidade e lista de arquivos que serão tocados **Quem aprova:** você — **portão humano obrigatório**

Regra inegociável: **a IA apresenta o plano e para. Não escreve código na mesma resposta.** Se o plano vier junto com 800 linhas implementadas, o portão não existiu — você vai aprovar retroativamente algo que já está pronto, que é o oposto de revisar.

Classificação de criticidade que decide o rigor das etapas seguintes:

Nível	Definição	Rigor exigido
CRÍTICO	Decide dinheiro, risco ou recomendação de investimento	G3 completo + golden tests + auditoria G7
IMPORTANTE	Afeta a leitura do usuário, mas não a decisão financeira direta	G3 simplificado + teste de borda
BAIXO	Apresentação, texto, layout	Validação visual basta

Em produto de derivativos, **quase tudo que exhibe número é CRÍTICO**. Se a IA classificar como BAIXO um cálculo de payoff, isso por si só é sinal de que ela não entendeu o domínio — pare e reveja.

Prompt: Anexo II, Prompt 2.

G3 — CONTRATO DE DADO ⚠ PORTÃO CRÍTICO

Skill: integridade-de-dado-exibido **Artefato de saída:** Mapa de Proveniência + **Prova de Fonte** **Quem aprova:** você

Este é o portão que teria interceptado 100% dos achados críticos da auditoria. Todos os sete nasceram do mesmo lugar: a tela foi construída antes de a fonte de dado existir, e a lacuna foi preenchida com valor plausível.

G3.1 — Prova de Fonte antes de qualquer tela

Regra: nenhuma interface é construída antes de existir um payload real da fonte, salvo em arquivo.

Na prática: antes de escrever o componente, a IA executa uma chamada real (`curl` , script, endpoint de teste) e grava a resposta em `docs/fontes/<endpoint>.sample.json` . Esse arquivo é o contrato. Se a chamada não funciona — credencial errada, endpoint inexistente, plano sem permissão — **isso é a tarefa**, e não se avança para a tela.

Foi exatamente aqui que o RADAR descarrilhou: a dependência `ws` está no `package.json` desde o primeiro commit e nunca foi usada. O streamer DXLink nunca foi ligado. A tela foi feita mesmo assim, com `mockOptions` .

G3.2 — Mapa de Proveniência

Tabela obrigatória, uma linha por número que aparece na tela:

Campo exibido	Fonte real	Endpoint / arquivo	Marca	Se a fonte falhar
Spot	Tastytrade	/quotes	MEDIDO	estado vazio + "cotação indisponível"
IV Rank	Tastytrade	/market-metrics	MEDIDO	estado vazio
VRP	cálculo	IV30 – RV20	DERIVADO	não exibe se um insumo faltar
Breakeven	cálculo	strikes + crédito	DERIVADO	não exibe
Score macro	—	—	SIMULADO	exige selo visível na tela

Regra de contágio: cálculo que consome `SIMULADO` produz `SIMULADO` . Nunca promova a marca.

Regra de rejeição: qualquer linha cuja coluna "Se a fonte falhar" contenha um número é rejeição imediata do plano. A resposta correta é sempre estado vazio, nunca preset.

Prompt: Anexo II, Prompt 3.

G4 — CONSTRUÇÃO

Skill: `padroes-engenharia-software` **Artefato de saída:** código + diff pequeno o suficiente para você ler

A regra que mais importa nesta etapa: fatiar

Uma fatia por vez, com revisão entre elas. Fatia = algo que você consegue ler em 10 minutos e rodar. O RADAR chegou a 10.152 linhas em 9 commits — em média 1.100 linhas por commit, com 3 commits sem descrição. Nesse volume, revisão humana não acontece: você aprova por confiança, não por leitura. E foi assim que sete defeitos críticos entraram sem resistência.

Ordem de construção obrigatória, e ela não é negociável em sistema de dado:

```
1. Adaptador de fonte → com o sample.json do G3 como teste de contrato
2. Motor de cálculo → função pura, com golden test
3. Rota / API → com auth, validação e rate limit desde a primeira versão
4. Interface → por último, consumindo o que já foi provado
```

A inversão dessa ordem — construir a tela primeiro e "ligar o dado depois" — é a causa-raiz mecânica de tudo que a auditoria encontrou. Tela pronta esperando dado **sempre** ganha um preset provisório, e provisório em software é permanente.

Segurança de rota desde a primeira versão

Não existe "coloço auth depois". `POST /api/avatar/generate` está público hoje, chamando um serviço pago, porque foi construído sem auth "para testar rápido". Toda rota nasce com: autenticação, validação de schema (Zod), rate limit e timeout.

Prompt: Anexo II, Prompt 4.

G5 — VALIDAÇÃO

Skills: `qa-sistemas-proprios` + `integridade-de-dado-exibido` **Artefato de saída:** suíte com golden tests e relatório de cobertura real

Golden test: o único teste que vale em cálculo financeiro

Para toda fórmula que decide dinheiro:

- 1. Calcule o resultado **à mão, fora do código**, e documente a conta no próprio teste
- 2. Asserte o valor numérico exato, não apenas que "é maior que zero"
- 3. Inclua **caso assimétrico**, não só o simétrico — fórmula que acerta por coincidência quebra silenciosamente depois

Exemplo do que faltava no RADAR:

```
it('breakeven do Iron Condor deriva dos strikes, nao de % do spot', () => {
  // Conta a mao: shortPut 135, shortCall 155, credito 2.00
  // BE inferior = 135 - 2.00 = 133.00 | BE superior = 155 + 2.00 = 157.00
  const r = volatilityEngine.evaluate(inputNVDA);
  expect(r.lowerBreakeven).toBe(133.00);
  expect(r.upperBreakeven).toBe(157.00);
});
```

O teste atual do motor de fundamentos afirma `expect(roeMetric?.value).toBe(16.5)` — onde 16,5 é um literal hardcoded no código de produção. Isso passa sempre e não pode falhar.

O teste do teste

Antes de aceitar qualquer teste escrito pela IA, faça uma pergunta:

Existe alguma alteração no código de produção que faria este teste falhar?

Se não existe, o teste é decorativo e deve ser reescrito. Aplique isso a toda suíte gerada por IA — é o filtro mais barato e mais eficaz que existe.

Prompt: Anexo II, Prompt 5.

G6 — DEPLOY

Skill: arquitetura-infraestrutura-deploy Artefato de saída: pipeline verde + plano de rollback escrito antes do deploy

Checklist específico para este tipo de projeto:

- ☐ Nenhum segredo em `.env` sem consumidor no código (hoje: `ANTHROPIC_API_KEY` órfã)
- ☐ Rotação de chave possível sem rebuild
- ☐ Cache de token **não** depende de filesystem (em serverless o disco é efêmero — o `tasty_token.json` não funciona na Vercel)
- ☐ Rotas públicas com rate limit no edge
- ☐ Plano de rollback escrito **antes**, não improvisado durante

G7 — AUDITORIA INDEPENDENTE

Sem skill de desenvolvimento. Sessão nova, contexto limpo, postura adversarial.

Esta etapa é o que produziu o laudo que você está corrigindo agora, e ela tem uma exigência estrutural:

A IA que construiu não pode auditar o que construiu.

Não por má-fé — por contexto. Na sessão em que ela construiu, o histórico contém as justificativas dela para cada escolha, e ela vai rere `mockOptions` como "decisão consciente de MVP" em vez de "dado falso em produção". Contexto limpo remove esse viés. É a mesma razão pela qual `qa-sistemas-proprios` manda separar o chapéu de quem construiu do de quem valida — mas aplicada à IA, ela precisa de **sessão separada**, não só de instrução.

Cadência recomendada:

Quando	Escopo
Fim de cada sprint	Diff da sprint, com foco em proveniência
Antes de exposição a terceiro	Auditoria completa (a que foi feita)
Trimestral	Completa, mesmo sem mudança relevante

Prompt: Anexo II, Prompt 6.

PARTE B — CORREÇÃO DO RADAR-TASYTRADE

Aplicação do processo acima ao sistema atual. As fases seguem o plano de remediação do laudo, agora com portão, skill e prompt definidos.

Sprint 0 — Contenção (24-48 h) — fazer hoje

#	Ação	Portão	Verificação
1	Password protection na Vercel ou tirar do ar	—	acessar em aba anônima e ser barrado
2	Revogar e reemitir DID_API_KEY e ANTHROPIC_API_KEY	—	chave antiga retorna 401
3	Banner global: "dados simulados, não use para decisão"	G4	visível em toda tela
4	Exibir o campo source que já existe no tipo	G3	selo por card

O item 4 é o de melhor relação custo-benefício do projeto inteiro: **a infraestrutura de proveniência já está construída e só não é exibida**. São poucas horas de UI para transformar o pior problema (dado falso disfarçado de real) no menor (dado falso declarado).

Sprint 1 — Fundação (semana 1)

Executar o **G0 completo** no repositório: CLAUDE.md, provenance.ts, ESLint, CI. Fazer isso **antes** de qualquer correção funcional — é o que impede a correção de reintroduzir a mesma classe de defeito.

Prompt de abertura: Anexo II, **Prompt 7**.

Sprint 2 — Fontes reais (semanas 2-3)

Ordem obrigatória, uma fonte por vez, cada uma com Prova de Fonte (G3.1) antes da tela:

- 1. **Cotação spot** → endpoint de quotes da Tastytrade. Eliminar os três datasets concorrentes; fonte única.
- 2. **Cadeia de opções** → /option-chains/{symbol}/nested + greeks do streamer. O gex-engine não muda uma linha — ele já está correto.
- 3. **OHLCV** → série real. **Se não houver fonte, a aba técnica sai do ar**. Não existe meio-termo aceitável: generateCandlesticks é deletado nesta sprint, com ou sem substituto.
- 4. **Eliminar todos os fallbacks numéricos** — os quatro pontos mapeados no laudo (\$A-14).

Sprint 3 — Matemática (semana 4, paralelizável)

Cada correção com golden test calculado à mão **antes** da correção do código:

- Breakeven derivado dos strikes reais
- Max Loss por asa, preparado para estrutura assimétrica
- POP calculado por delta, não constante
- Prêmios do book real — sem book, a estrutura não é recomendada
- Regra do crédito corrigida para 1/3 (0,3333), não 0,30
- Unificar as duas convenções incompatíveis de símbolo OCC

Sprint 4 — Governança (semana 5)

- Remover os overrides de VALE3 dos três arquivos
- Reescrever os testes tautológicos
- Fechar as rotas: auth + Zod + rate limit
- Corrigir o README (hoje afirma "100% de cobertura" e "IA em tempo real", ambos falsos)

Sprint 5 — Reauditoria (semana 6)

G7 completo, sessão limpa. Critério de aceite: **zero achado crítico e zero achado alto**. Médios e baixos podem virar backlog; críticos e altos, não.

ANEXO I — MODELO DE CLAUDE.md

Salvar como CLAUDE.md na raiz do projeto. Este arquivo carrega em toda sessão automaticamente — é onde ficam os invariantes que não podem depender de a skill certa disparar.

```
# RADAR TASTYTRADE — Instruções de Projeto

## Natureza do sistema
Terminal quantitativo de apoio à decisão em derivativos (Tastytrade, mercado US).
Números exibidos aqui embasam decisão de alocação de capital real.
Criticidade padrão de qualquer coisa que exiba número: CRÍTICO.
```

```
## INVARIANTES — violação bloqueia entrega, sem exceção

1. NÃO INVENTE NÚMERO. Se a fonte de dado não está ligada ou falhou, a tela mostra estado vazio com motivo. Nunca preset, nunca "valor razoável", nunca constante por ticker.
2. `Math.random()` é proibido em `src/lib/domain/**` e `src/lib/services/**`.
3. `catch {}` vazio é proibido. Falha retorna null + motivo, nunca valor default.
4. Todo número exibido carrega marca de proveniência visível na UI: MEDIDO / DERIVADO / ESTIMADO / SIMULADO. Cálculo que consome SIMULADO produz SIMULADO — nunca promova a marca.
5. Fórmula que decide dinheiro (payoff, breakeven, max loss, POP, prêmio) deriva dos parâmetros reais da estrutura. Percentual fixo do spot é proibido.
6. Rota de API nasce com autenticação, validação Zod, rate limit e timeout. Não existe "coloco auth depois".
7. Fonte única por campo. Dois lugares com o preço do mesmo ativo é defeito.

## ORDEM DE CONSTRUÇÃO (não inverter)
adaptador de fonte → motor de cálculo → rota → interface
Tela nunca é construída antes de existir payload real salvo em `docs/fontes/`.

## FLUXO DE TRABALHO
- Mudança estrutural: apresente o PLANO e PARE. Não escreva código na mesma resposta.
- Uma fatia por vez, revisável em 10 minutos.
- Toda feature que exhibe número entrega o Mapa de Proveniência junto do código.

## O QUE NUNCA AFIRMAR
Não rotule dado como "ao vivo", "tempo real" ou "oficial" se qualquer campo da tela for estático. Não declare cobertura de teste sem relatório medido. Não descreva no README capacidade que o código não tem.

## Stack
Next.js 14 (App Router) · TypeScript strict · Tailwind · Vitest
Fontes: Tastytrade API (OAuth2 + DXLink) · BRAPI (ativos BR)
```

ANEXO II — PROMPTS PRONTOS

Prompt 1 — Abertura de requisito (G1)

Use a skill analista-funcional-requisitos.

Preciso de: [descreva a necessidade em linguagem de negócio, não como solução]

Antes de propor qualquer solução técnica, responda por escrito:

1. Que problema real isso resolve
2. Qual a regra de negócio, em linguagem de negócio
3. Quais as exceções
4. O que acontece com o usuário quando falha
5. DE ONDE VEM CADA DADO que a tela vai mostrar — fonte, endpoint, e o que acontece se essa fonte estiver indisponível

Se algum item ficar sem resposta, pergunte. Não preencha lacuna com suposição. Não escreva código nesta resposta.

Prompt 2 — Plano técnico (G2)

Use as skills papel-engenheiro-senior e padroes-engenharia-software. Leia o CLAUDE.md do projeto antes de responder.

Requisito aprovado: [cole a saída do Prompt 1]

Entregue APENAS o plano, sem código:

- Classificação de criticidade (CRÍTICO / IMPORTANTE / BAIXO) e a justificativa
- Arquivos que serão criados ou alterados, com o motivo de cada um

- Fatiamento: quebre em entregas de no máximo 10 minutos de revisão cada
- Riscos e o que você faria diferente do que eu pedi, se for o caso
- O que fica FORA do escopo

Pare aqui e aguarde minha aprovação. Não implemente nada nesta resposta.

Prompt 3 — Contrato de dado (G3) ⚠ o mais importante

Use a skill integridade-de-dado-exibido.

Antes de escrever qualquer linha de interface:

1. PROVA DE FONTE: execute uma chamada real a cada fonte que esta feature consome. Salve o payload em docs/fontes/<endpoint>.sample.json.
Se a chamada falhar (credencial, permissão de plano, endpoint inexistente), PARE e me reporte. Resolver isso É a tarefa. Não prossiga para a tela.
2. MAPA DE PROVENIÊNCIA: tabela com uma linha por número que aparecerá na tela:
| Campo | Fonte real | Endpoint | Marca | Se a fonte falhar |

Regras:
 - Marca ∈ {MEDIDO, DERIVADO, ESTIMADO, SIMULADO}
 - Cálculo que consome SIMULADO produz SIMULADO
 - A coluna "se a fonte falhar" NUNCA contém um número. Só estado vazio ou erro.
3. Aponte todo campo que hoje você não tem como preencher com dado real.
Prefiro tela com menos campos e todos verdadeiros a tela completa com estimativa.

Pare e aguarde aprovação antes de construir.

Prompt 4 — Construção (G4)

Use as skills papel-engenheiro-senior e padroes-engenharia-software.
CLAUDE.md e Mapa de Proveniência aprovado valem como contrato.

Implemente APENAS a fatia [N] do plano aprovado.

Ordem obrigatória: adaptador de fonte → motor → rota → interface.
Nesta fatia estamos em: [etapa]

- Restrições:
- Nenhum valor numérico literal em código de domínio
 - Falha de fonte retorna null + motivo, nunca valor default
 - Rota nasce com auth + Zod + rate limit + timeout
 - Ao final, liste o que você NÃO conseguiu ligar em dado real e por quê

Não avance para a próxima fatia sem minha revisão.

Prompt 5 — Validação (G5)

Use as skills qa-sistemas-proprios e integridade-de-dado-exibido.

Para cada fórmula que decide dinheiro nesta fatia:

1. Calcule o resultado à mão, fora do código, e documente a conta no teste
2. Asserte o valor numérico exato – não "maior que zero"
3. Inclua caso assimétrico, não só o simétrico
4. Inclua caso de fonte ausente e fonte corrompida

Depois, aplique o teste do teste a cada teste que você escreveu:
"existe alguma alteração no código de produção que faria este teste falhar?"
Se a resposta for não, o teste é decorativo – reescreva.

Não afirme cobertura sem relatório medido de coverage.

Prompt 6 — Auditoria independente (G7) — sessão nova, sempre

Você é um auditor sênior de sistemas com mais de 20 anos de experiência. Não participou da construção deste sistema e não tem compromisso com nenhuma decisão tomada nele.

Audite [caminho do projeto] nos eixos: qualidade, integridade, acuracidade e segurança dos dados.

Foco prioritário – para cada número que o sistema exibe ao usuário, responda:

- Ele foi medido de uma fonte real, ou está hardcoded / gerado / estimado?
- O usuário consegue saber qual dos dois, olhando a tela?
- Se a fonte falhar, o que aparece no lugar?

Verifique também:

- Fórmulas financeiras contra o valor correto calculado independentemente
- Testes tautológicos (que passam sempre e não podem falhar)
- Rotas de API sem autenticação, rate limit ou validação
- Segredos: versionados, órfãos, ou expostos em bundle
- README e textos de UI que afirmam capacidade que o código não tem

Classifique cada achado por severidade, cite arquivo e linha, e feche com recomendação clara: remediar ou reescrever, com estimativa. Não suavize achado por consideração ao trabalho já feito.

Prompt 7 — Abertura da correção do RADAR (Sprint 1)

Use as skills papel-engenheiro-senior, padroes-engenharia-software e integridade-de-dado-exibido.

Contexto: o sistema passou por auditoria independente que encontrou 23 achados, 7 críticos, quase todos da mesma classe – dado sintético apresentado como real. O laudo está em AUDITORIA_RADAR_TASTYTRADE.pdf, na raiz do projeto. Leia antes.

Esta primeira tarefa NÃO corrige nenhum defeito funcional. Ela instala as barreiras que impedem a reintrodução da mesma classe de defeito:

1. CLAUDE.md na raiz com os invariantes do projeto
2. src/lib/types/provenance.ts com o tipo Tracked<T>
3. Regras de ESLint proibindo Math.random() e catch vazio em domain/ e services/
4. .github/workflows/ci.yml bloqueando merge sem tsc + lint + test + build

Ao final, demonstre que a barreira funciona: escreva de propósito um Math.random() em domain/ e me mostre o lint falhando.

Não corrija nenhum outro achado do laudo nesta tarefa.

ANEXO III — CARTÃO DE PAREDE

As sete regras que, se seguidas, teriam evitado 23 dos 23 achados:

1. **Fonte antes de tela.** Payload real salvo em arquivo antes de existir componente.
2. **Sem fonte, sem número.** Estado vazio com motivo, nunca preset.
3. **Proveniência visível.** MEDIDO / DERIVADO / ESTIMADO / SIMULADO em toda tela.
4. **Fórmula de dinheiro deriva dos parâmetros reais,** nunca de percentual do spot.
5. **Plano aprovado antes de código.** A IA propõe e para.
6. **Fatia revisável em 10 minutos.** Diff grande não é revisado, é confiado.
7. **Quem constrói não audita.** Sessão limpa, postura adversarial, cadência fixa.

Este processo pressupõe que as barreiras do G0 existam. Sem elas, os portões G1–G7 dependem exclusivamente de disciplina, e disciplina perde de prazo. A ordem de implantação — G0 antes de qualquer correção funcional — não é preferência metodológica: é a diferença entre corrigir os defeitos e corrigir a causa deles.